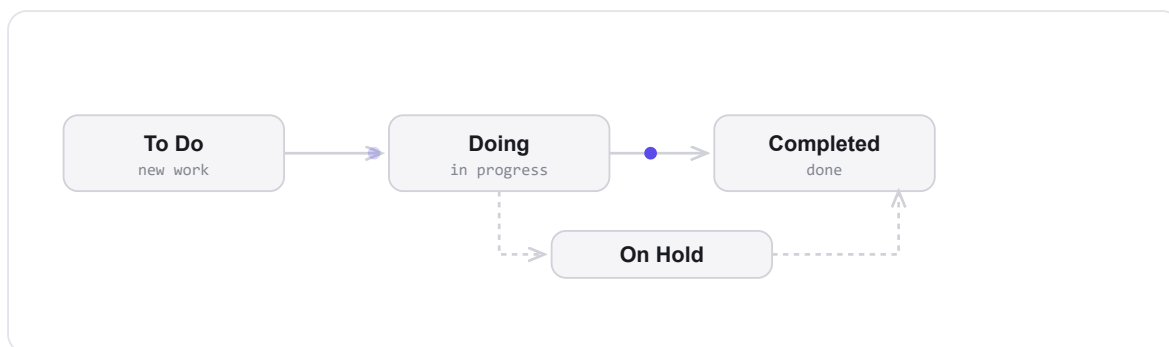


Pyramid

A task and project workspace built with Next.js and NestJS, backed by MongoDB Atlas. This document walks the application screen by screen, traces how a change reaches the database, and records what was verified — and what was not.



43 assertions passing	0 console errors
20 improvements identified	10 widths verified

Janmejy Mahato 21 August 2026 Next.js 16 • NestJS 11 • MongoDB

CONTENTS

- 01 What this application does
- 02 Architecture
- 03 Authentication
- 04 The task list
- 05 Board view
- 06 Search and filtering
- 07 Task detail
- 08 Projects
- 09 Settings and theming
- 10 Responsive behaviour
- 11 How a change persists
- 12 Verification
- 13 Improvements identified and implemented
- 14 Known limitations
- 15 Running locally

Short on time? Section 12 carries the verification results and section 13 the twenty improvements identified — those two answer most questions about what was built and what was found.

01

What this application does

Pyramid is a workspace for tracking tasks and projects. Work moves through four stages, and the same data is presented two ways — grouped tables and Kanban columns — with every task opening into a detail page carrying description, labels, subtasks, comments and an activity feed.

Next.js 16

React 19

TypeScript

Tailwind v4

NestJS 11

MongoDB

Mongoose 9

JWT

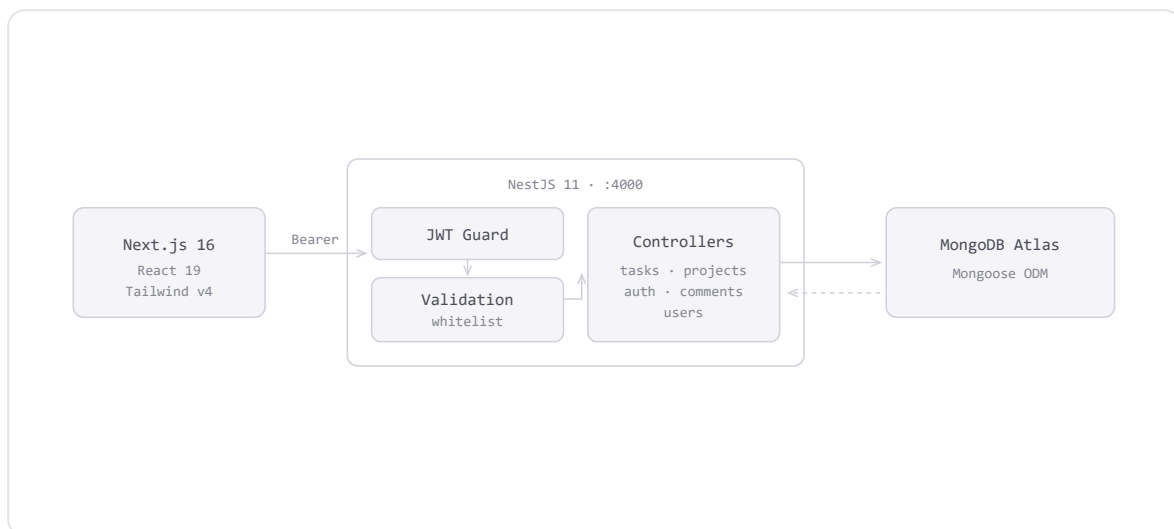
Google OAuth

- © **The governing constraint:** nothing is a local-only illusion. Every visible control writes through to the database — change a priority, reload the page, and the change is still there. Section 12 records the test run that proves this across all 21 interactive paths.

02

Architecture

Two packages — a Next.js frontend and a NestJS API — talking REST, with MongoDB Atlas for storage. Requests cross three gates before touching data: token verification, DTO validation, then the service layer.



Decisions worth naming

One grouped endpoint feeds both views. `GET /api/tasks/grouped` returns tasks already bucketed by status. List and board render the same payload, so the two can never disagree, and switching between them costs no request.

Mutations re-fetch instead of patching local state. Every write calls the API then reloads the grouped payload — trading a little latency for the guarantee that what you see is what the database holds.

The vocabulary is defined once per side. Four statuses, five priorities, declared in `constants.ts` and enforced at the validation boundary. Note the deliberate casing split: statuses are Title Case (`"To Do"`), priorities lowercase (`"high"`).

Theming is two independent axes. `data-theme` and `data-accent` are separate attributes driving CSS custom properties. No component hardcodes a colour.

03

Authentication

Continue as Guest calls `POST /api/auth/guest`, which creates a throwaway user document and returns a signed JWT. The token lands in `localStorage`; on each subsequent load `AuthProvider` resolves it via `GET /auth/me`. An expired or revoked token is discarded and the app falls back to anonymous rather than erroring.

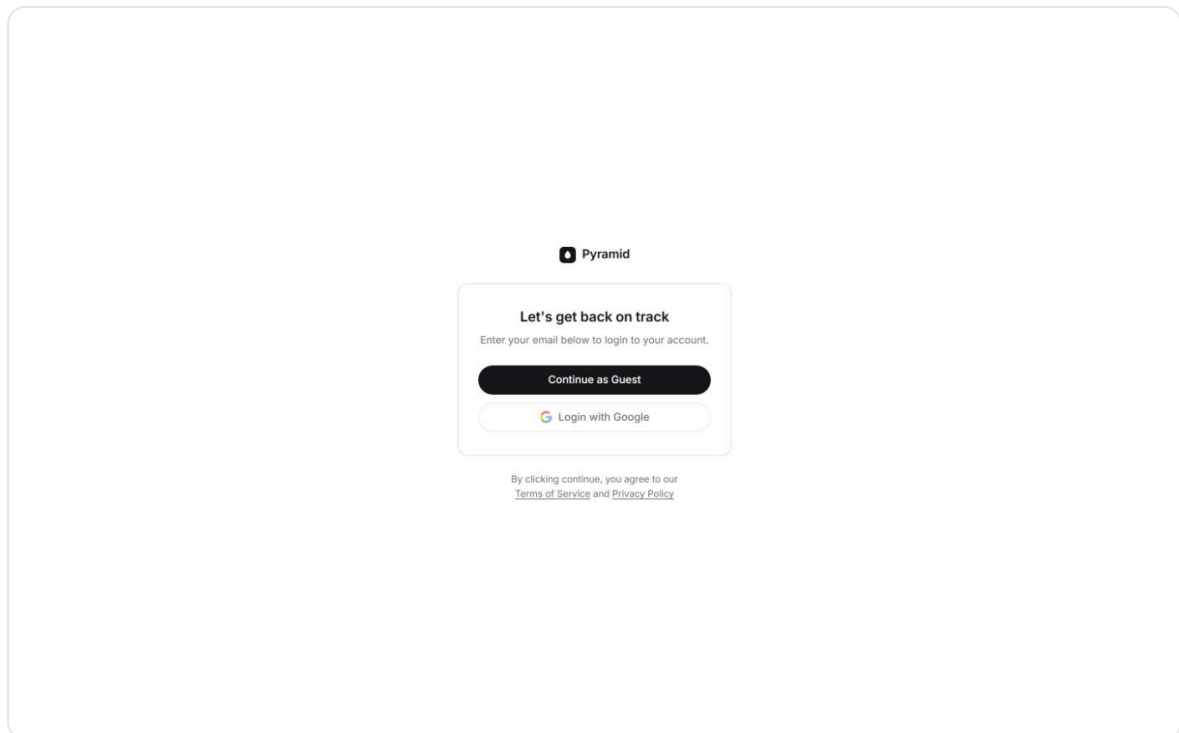
Each guest login creates a *distinct* user, so two people testing never share a board.

Google sign-in

The Google button runs the OAuth 2.0 authorization-code flow and ends in the same kind of session token. The token comes back in a **URL fragment** rather than a query string — fragments are never sent to a server, so it stays out of access logs and `Referer` headers.

The CSRF state is an **HMAC signed with** `JWT_SECRET` rather than a server-side session, so the flow survives a restart or a second instance. Accounts match on Google's stable subject id rather than email, and an unverified email is never used for matching — otherwise an account could be claimed by signing up with its address.

Where credentials are not configured, `GET /auth/providers` reports it and the UI hides the button, so the app runs guest-only rather than offering a control that fails on click.



The entry screen. Both sign-in methods are live; the Google button hides itself when the server has no OAuth credentials configured.

04

The task list

The default landing view. Tasks are grouped into one table per status, each with its own header and its own **Add Task** affordance. Priority renders as a coloured glyph plus label, the colour carrying the semantics.

The screenshot shows the Dexter project management interface. On the left is a sidebar with 'Workspace' and 'Tasks' (selected). The main area displays a list of tasks organized into three status groups: 'To Do', 'Doing', and 'Completed'. Each group contains a table of tasks with columns: Task, Priority, Members, Due Date, and Actions. The 'To Do' group has three tasks: 'Write API Documentation' (High priority), 'Implement Search Function' (Medium priority), and 'Deploy to Production' (High priority). The 'Doing' group has two tasks: 'Code Review Completed' (Medium priority) and 'Design Mockups Finalized' (High priority). The 'Completed' group has three tasks: 'Feature Testing Passed' (Low priority), 'UI Design Updated' (Medium priority), and 'Security Audit Scheduled' (Urgent priority). Each task row includes an 'Add Task' button and an 'Actions' menu (three dots).

Task	Priority	Members	Due Date	Actions
Write API Documentation	High	1	29 Jul 2026	...
Implement Search Function	Medium	1	29 Jul 2026	...
Deploy to Production	High	1	29 Jul 2026	...
+ Add Task				
On Hold				

Four status groups, each an independent table. Optional columns are controlled from the Fields menu.

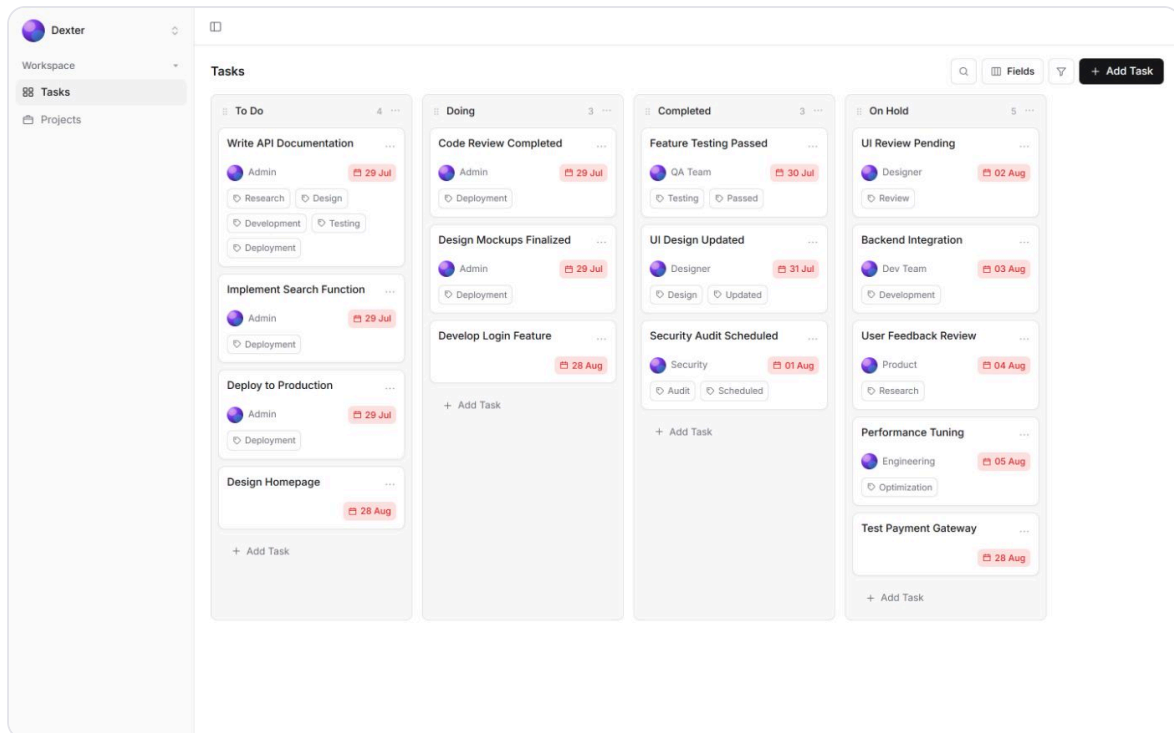
Inline creation. *Add Task* opens a text field in place; Enter creates the task in that group's status with a due date defaulted one week out, so a new row is never missing a date.

Row actions. The ... menu exposes priority, status and delete. Changes write immediately and relocate the row if its group changed.

05

Board view

The same four statuses as columns, each showing a count. Cards surface more than list rows do: assignee, a due-date chip, and labels as individual chips. The due-date chip carries a red-tinted background — this design treats due dates as attention-carrying, not neutral metadata.

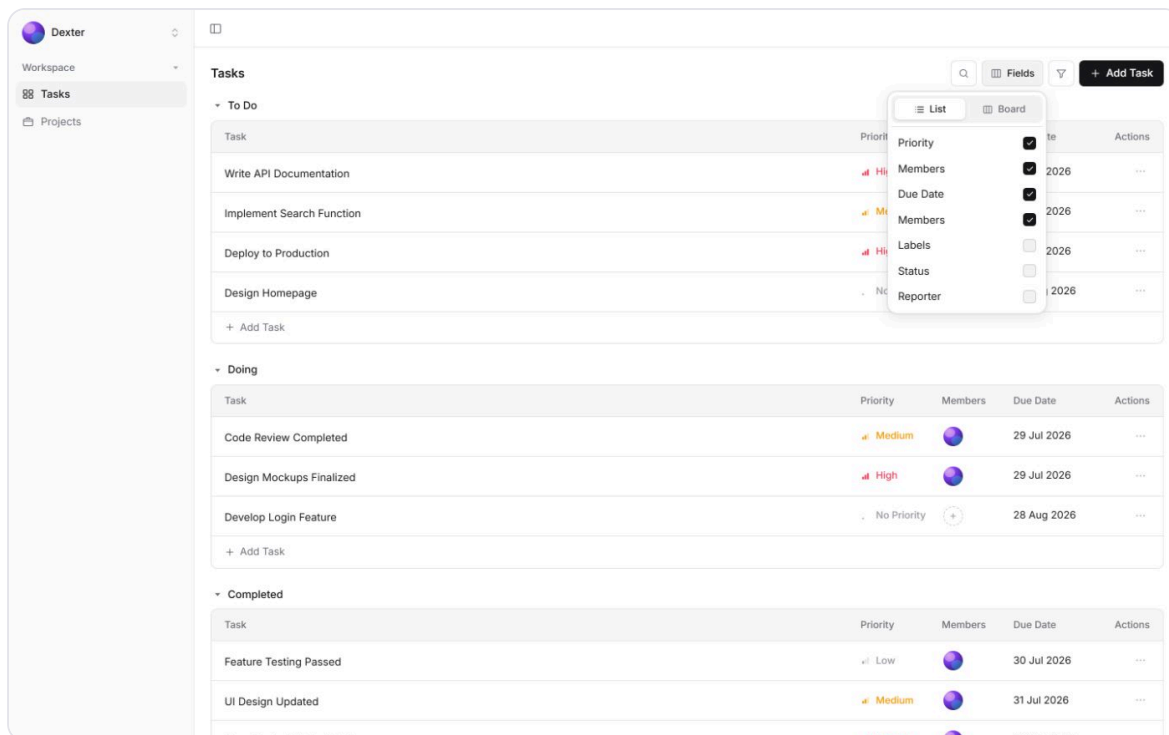


Board view. Same payload as the list, no refetch on switch.

06

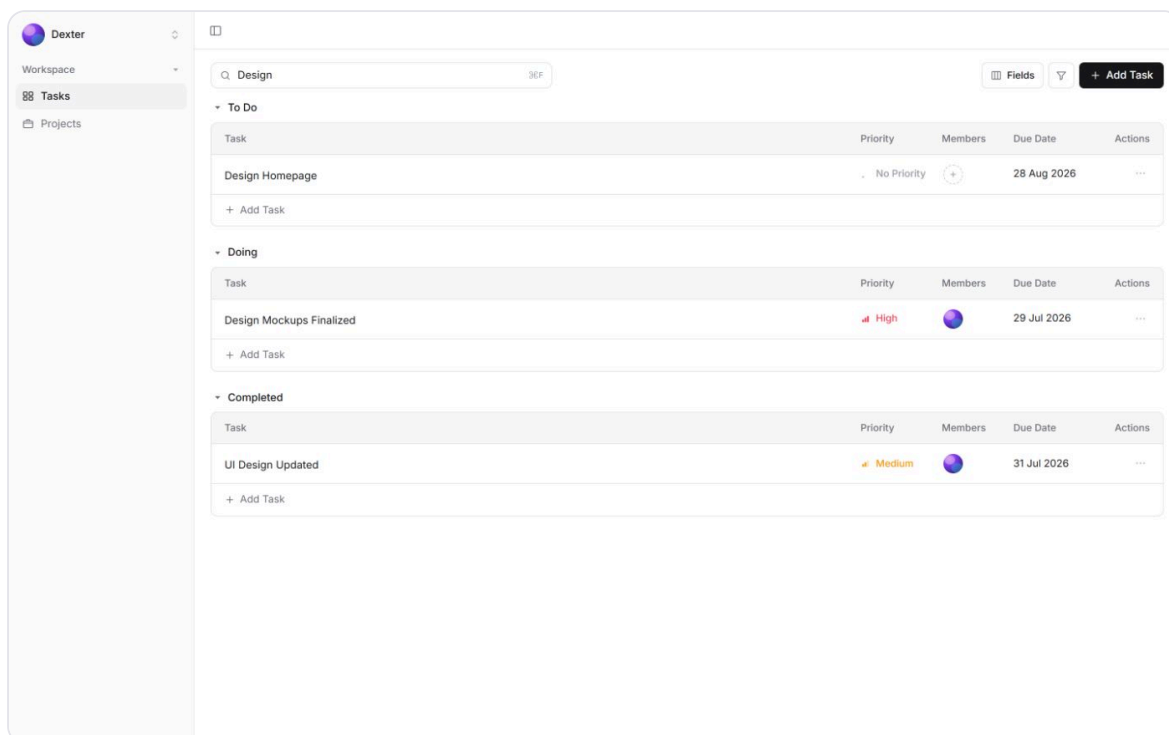
Search and filtering

Fields carries both the List/Board switch and per-column visibility.



The Fields menu. Note the two rows both labelled Members – see improvement 4.

Search opens from the magnifier or `⌘F` / `Ctrl+F`, intercepted so the app's search opens instead of the browser's. Input is debounced, and filtering runs client-side against the already-grouped payload, so results are instant.

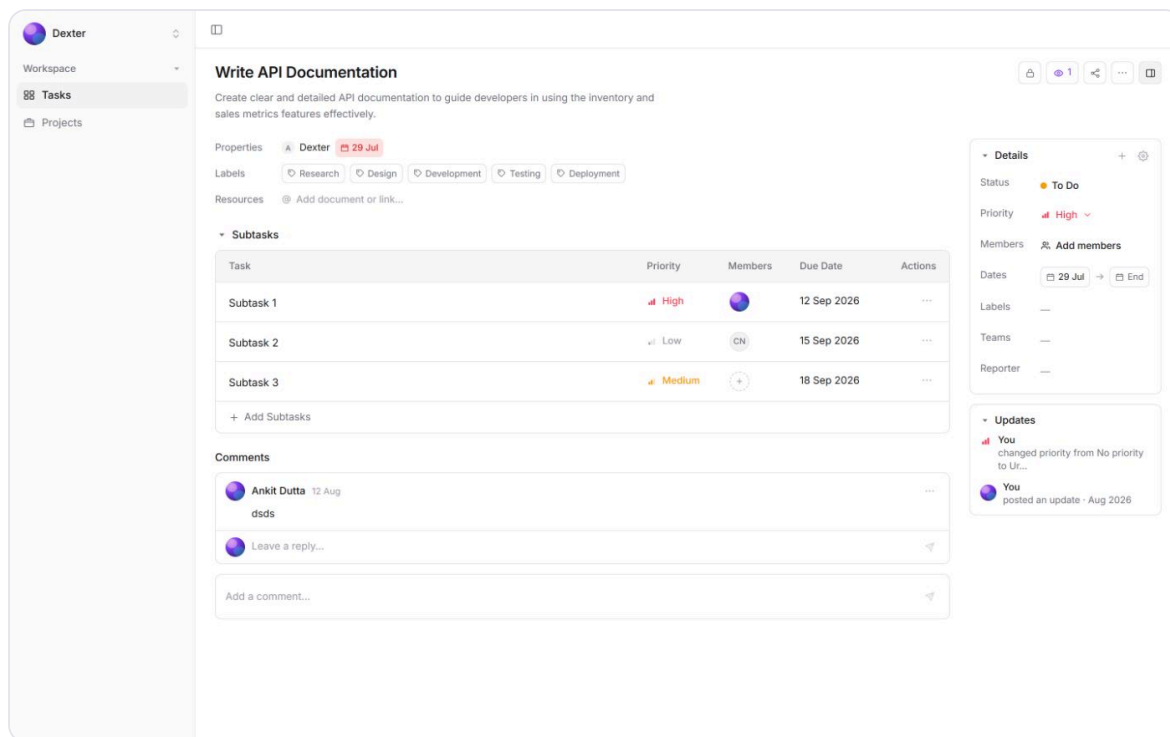


Search active – the page title collapses into the input. Empty groups hide while filtering.

07

Task detail

A two-column layout: content left, **Details** panel right. Title and description are click-to-edit, saving on blur — there is no save button because the design shows none.



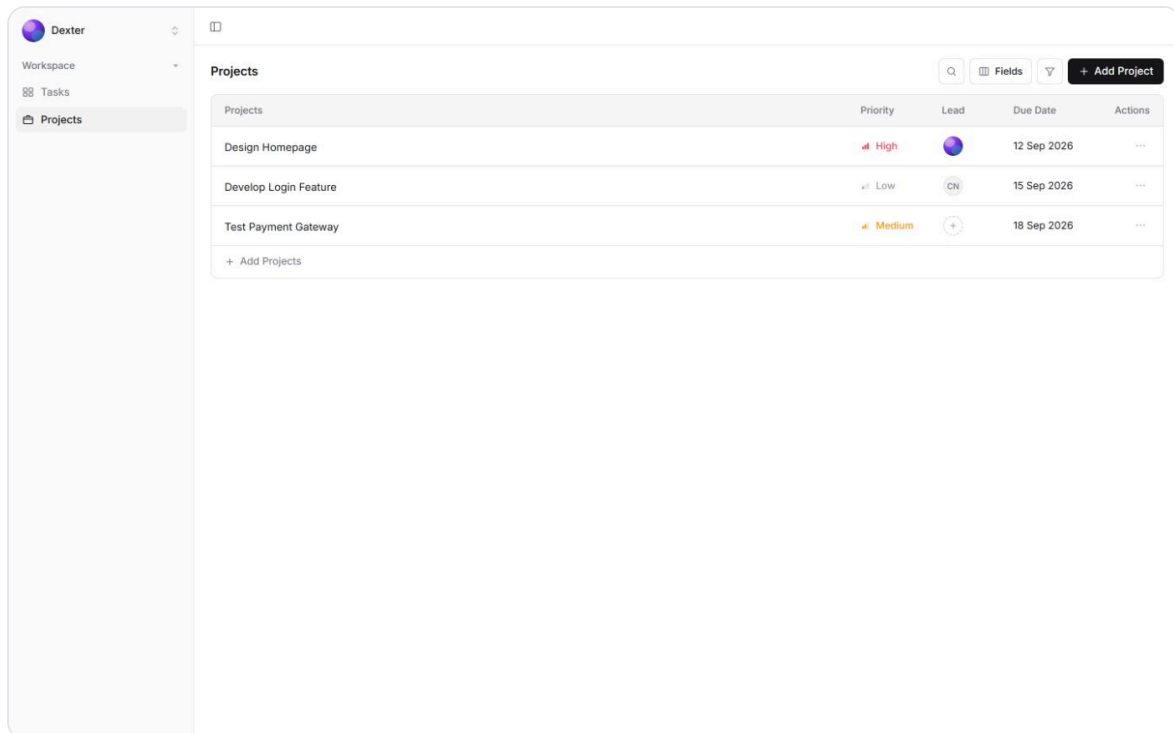
Task detail, captured after the improvement-1 fix – the subtask table and comment thread now carry distinct headings.

Subtasks reuse the same table component as the main list, so editing behaves identically at both levels. **Comments** support threaded replies; the composer retains its text if a post fails. **Updates** shows a server-recorded activity feed.

08

Projects

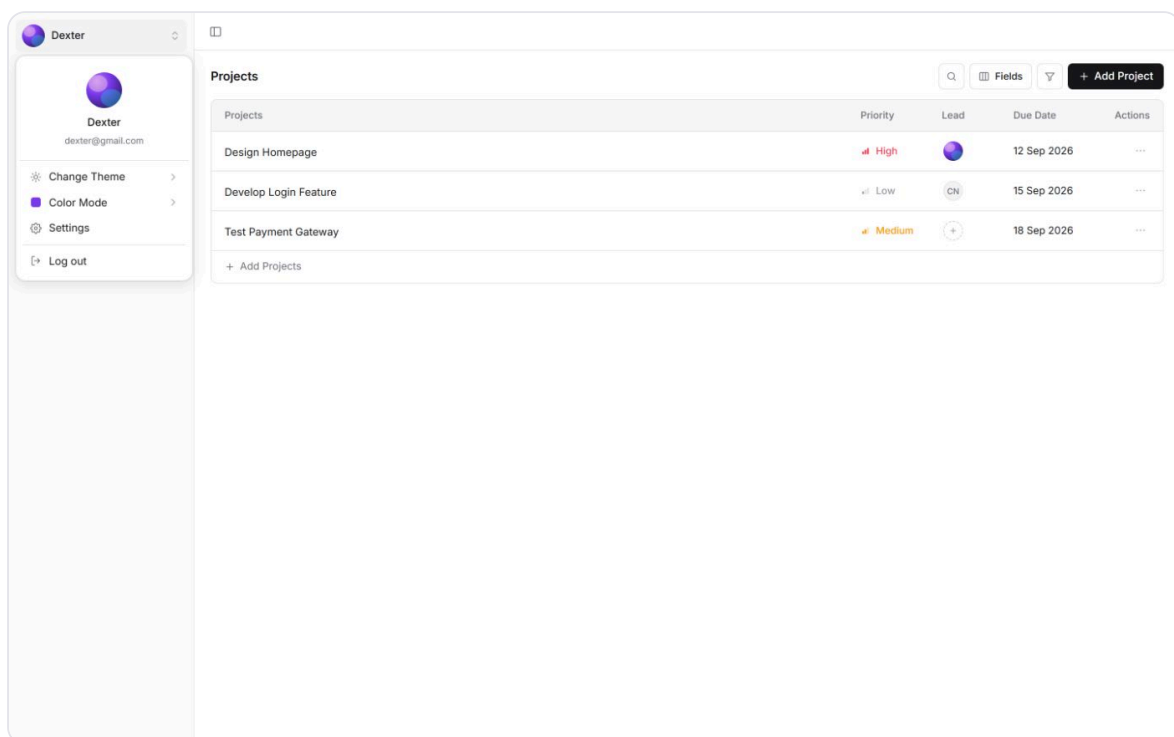
The same table shell as tasks, with Priority, Lead and Due Date. Each project opens to a detail page listing that project's tasks — the task list scoped by `projectId`.



Projects. The Lead cell shows all three member states: avatar, initials fallback (CN), and unassigned.

09

Settings and theming



The user menu – theme, colour, settings, logout.

Profile fields save on blur through `PATCH /api/users/me`, verified to survive a reload. Theme switches light/dark; Color picks among six accents.

← Back to app

Q Search

Profile

Theme

Color

Profile

Profile picture

Email

Full name Dexter

Title
Your job title or role

Username
One word, like a nickname or first name

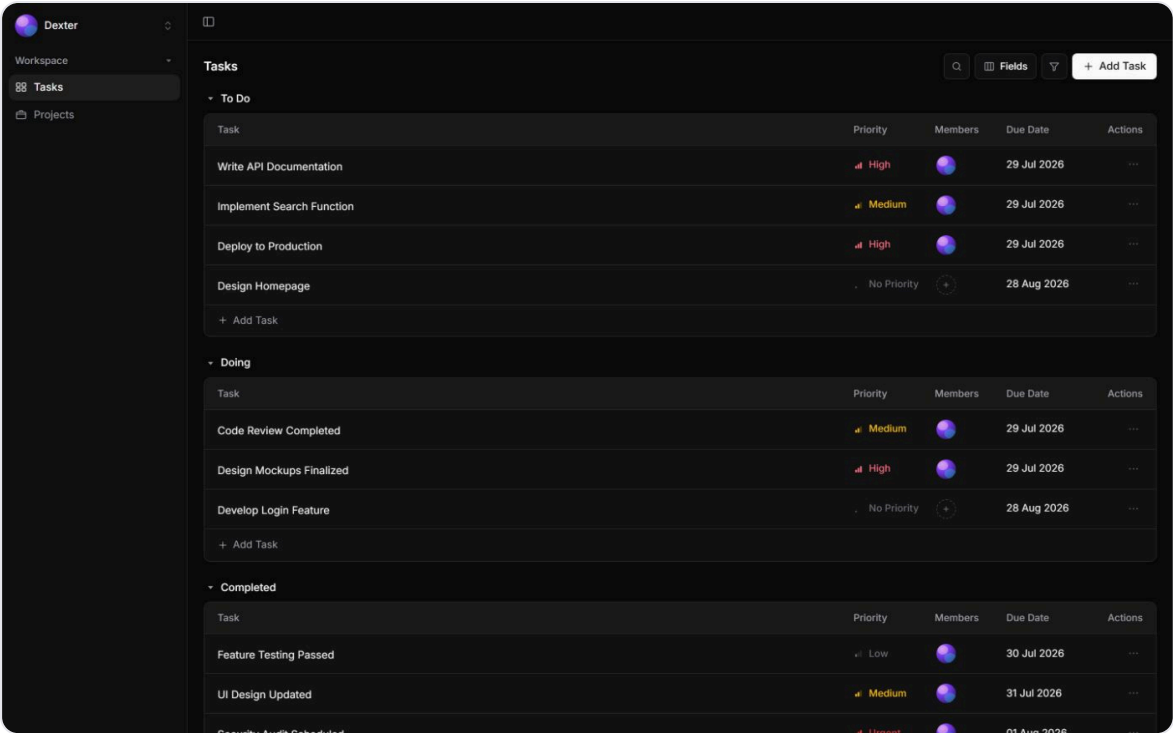
Workspace access

Remove yourself from the workspace

Leave Workspace

Settings. Editable fields with no save button, matching the design.

Dark mode is a genuine re-themed palette rather than an inversion — surfaces, borders, table headers and priority colours all carry separate dark values, with priority tones brightened for contrast.



Dark mode. Priority hues shift lighter to hold contrast against the dark ground.

10

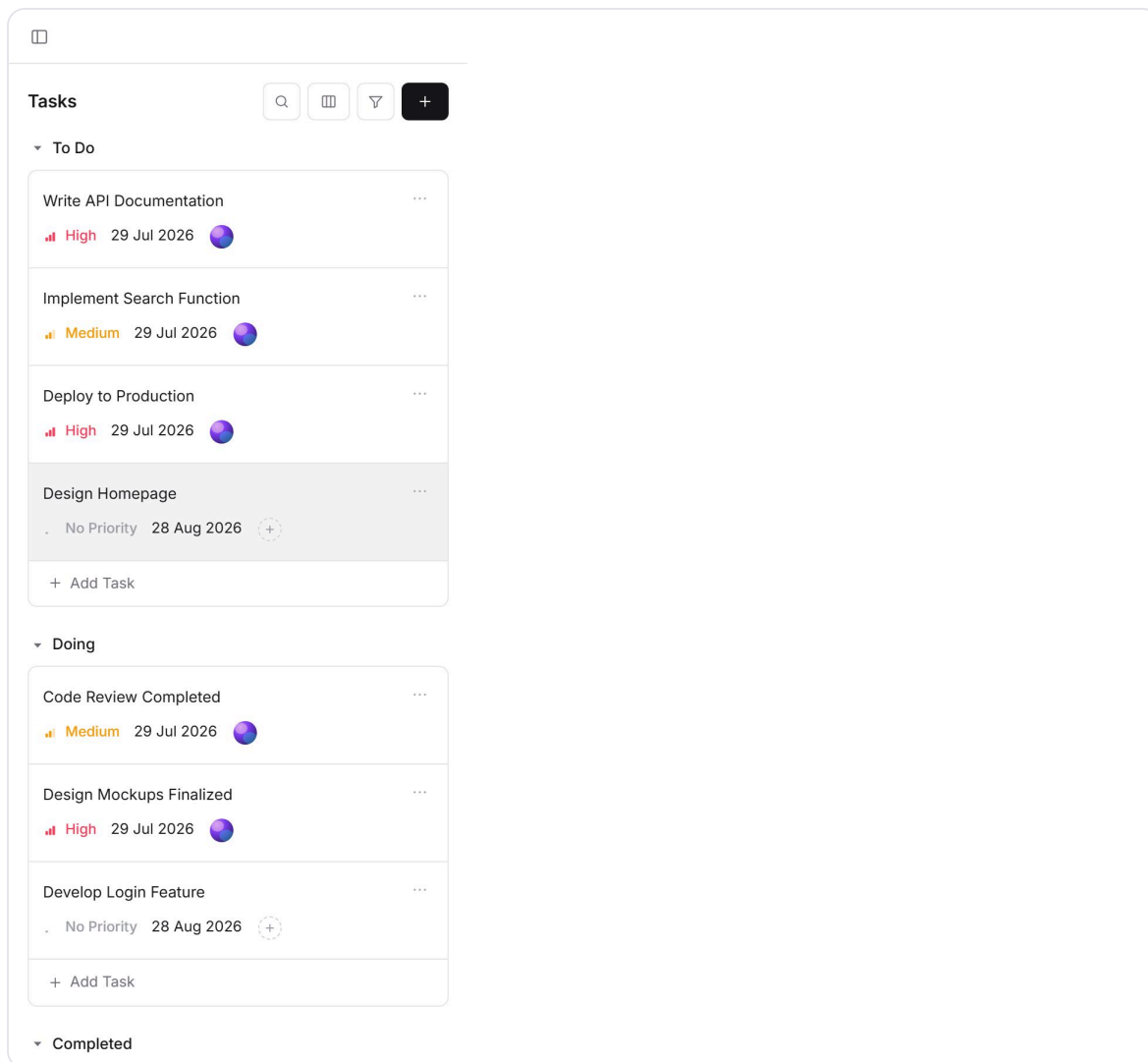
Responsive behaviour

Verified at five widths by comparing `scrollWidth` against `clientWidth`. No horizontal overflow at any breakpoint.

WIDTH	LAYOUT BEHAVIOUR	OVERFLOW
1440	Full two-pane, sidebar 228px	None
1280	Same, sidebar 210px	None
1024	Same, toolbar labels intact	None
768	Sidebar becomes overlay drawer	None
390	Tables restructure into stacked cards	None

The mobile treatment is the notable one. Rather than horizontally scrolling a five-column table, each row becomes a card — title on one line, priority, date and

assignee on a second. Toolbar buttons drop their labels; the sidebar becomes a drawer with scrim, scroll lock and Escape-to-close.

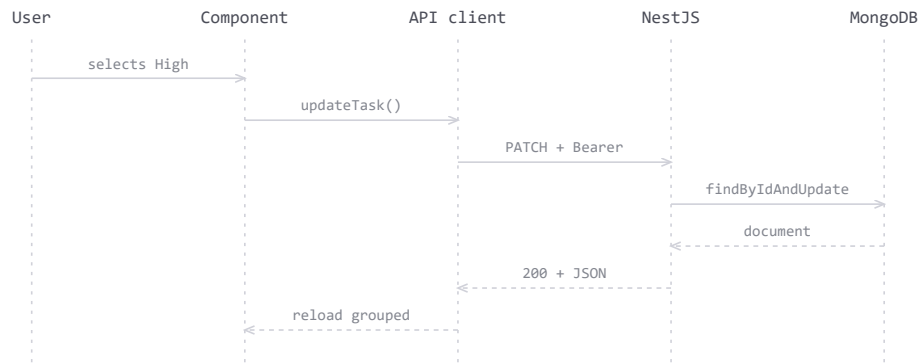


390px. The table restructures rather than scrolling sideways.

11

How a change persists

Changing a priority from a row menu, end to end:



1 The component calls the API client

No local state is changed yet — the UI still shows the old value.

2 Two gates run server-side

The JWT guard verifies the bearer token, then the ValidationPipe rejects unknown fields outright rather than ignoring them.

3 Mongo writes, and activity is recorded

The change and its audit entry land together.

4 The grouped payload is re-fetched

This is the important step: one endpoint feeds both list and board, so the two views can never drift apart.

- © **Why re-fetch instead of patching local state?** It trades a little latency for the guarantee that what you see is what the database holds — and removes the need for per-view optimistic-update logic entirely.

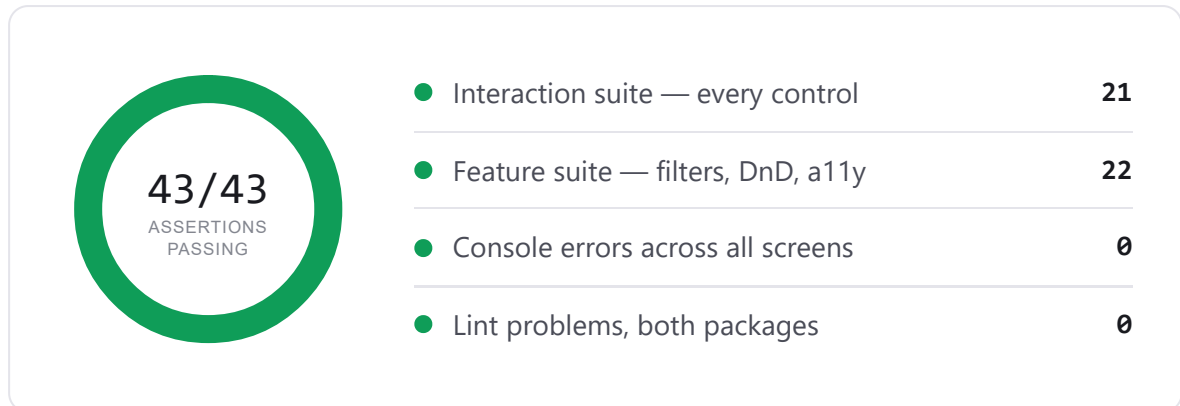
The validation boundary

A global `ValidationPipe` runs with `whitelist` and `forbidNonWhitelisted`, so an unknown field is a 400 rather than a value that silently does nothing. Errors flow through `AllExceptionsFilter` into a consistent shape, and the client's `ApiError` carries the status code so callers can tell 401 from 400.

12

Verification

Every figure below was produced by running the checks, not estimated.



CHECK	RESULT
Frontend TypeScript <code>tsc --noEmit</code>	Clean
Backend TypeScript	Clean
Frontend production build, 8 routes	Success
Backend build <code>nest build</code>	Success
ESLint, frontend and backend	0 problems

ENDPOINT	EXPECTED	RESULT
<code>GET /api/health</code>	200	200
<code>GET /api/tasks</code> unauthenticated	401	401
<code>POST /api/auth/guest</code>	201	201
CRUD round-trip — create → confirm 404	—	Passed

Tested against a live MongoDB Atlas connection, not a mock.

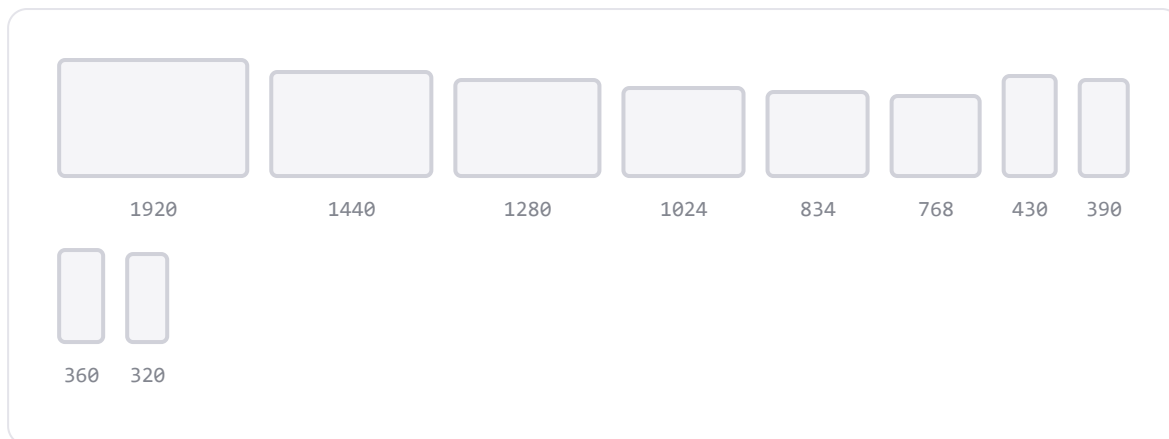
```

=== login ===
    PASS guest login navigates to /tasks
    PASS seeded data renders from MongoDB
=== create task ===
    PASS Add Task opens an inline field
    PASS new task appears
=== row actions ===
    PASS row actions menu opens
    PASS priority change persists
    PASS status move relocates the task
=== board view ===
    PASS board renders columns
    PASS task visible on board
=== task detail ===
    PASS navigates to task detail
    PASS title edit persists
    PASS description edit persists
=== details panel ===
    PASS status dropdown opens
    PASS members picker opens
=== subtasks + comments ===
    PASS subtask created
    PASS comment posted
=== projects ===
    PASS project created
    PASS project deleted
=== settings + theme ===
    PASS profile name persists after reload
    PASS theme switch applies
=== cleanup ===
    PASS task deleted

passed: 21  failed: 0
console errors: none

```

Each assertion drives a real control in Chromium **and verifies the change persisted through the API** — not merely that the UI updated locally.



Ten widths × four routes, measured by comparing `scrollWidth` against `clientWidth`. **No horizontal overflow anywhere.** Hover a frame to confirm.

13

Improvements identified and implemented

Twenty issues were identified across two review passes. **Eight have been implemented**; the remaining twelve are recorded with severity and the reasoning behind deferring them. Each entry states what was wrong, what changed, and how it was verified.



Part A — Implemented

8 items

Shipped and verified against the running application.

A1 • DEFECT FOUND DURING WALKTHROUGH — IMPLEMENTED

The comments section was labelled “Subtasks”

On task detail, the comments heading repeated the label of the subtask table directly above it. The surrounding code comment already read `{/* Comment thread */}`, confirming the label was wrong rather than intentional.

`task-detail-view.tsx:368` — **Changed:** the heading now reads “Comments”. Verified against the live DOM: exactly one `Subtasks` heading and one `Comments` heading.

A2 **Board drag-and-drop** IMPLEMENTED

Cards move by pointer drag from the grip handle, with a target-column highlight, drop-zone hint and cursor ghost — built on Pointer Events, since HTML5 drag-and-drop gives no usable touch drag and no keyboard path. A **full keyboard equivalent** ships alongside: Space to pick up, ←/→ to move, Space to drop, Esc to cancel, with aria-grabbed and a live region narrating the target. **Verified:** a card picked up, moved and dropped into the adjacent column.

A3 **All seven filter axes functional** IMPLEMENTED

Previously only Priority worked; the rest opened empty submenus. Status, Priority, Members, Due Date, Teams, Labels and Reporter now all filter — axes combine with AND, values within an axis with OR. Options derive from the loaded payload, so the menu only offers values that exist. **Teams** correctly reports “No teams”: the API has no teams field, so an empty state is the truthful rendering.

A4 **Duplicate “Members” label** IMPLEMENTED

Two Fields entries shared a label while toggling different columns. The second is now labelled “Assignees”.

A5 Undo on delete IMPLEMENTED

Deletions raise a toast with a six-second undo, recreating the record with its original priority, due date, members and labels — less interruptive than a confirmation dialog, and it recovers the more common regret. **Known incompleteness:** undo does not restore subtasks or comments — see D1 below.

A6 Mutation failures surfaced IMPLEMENTED

The change handlers previously awaited without catching, leaving a failed request unexplained. Every mutation now reports failure through a danger toast naming the action.

A7 Loading skeletons IMPLEMENTED

Table, board, projects and detail skeletons render at the real row heights, so arriving content does not shift the layout — most visible during the ~20s Atlas cold start.

A8 Focus rings and drawer focus trap IMPLEMENTED

A single `:focus-visible` treatment now covers links, buttons, menu items and inputs — keyboard only. The mobile drawer is a true `role="dialog"` with `aria-modal`, trapping Tab and restoring focus on close. **Verified:** ten consecutive Tab presses stay inside the drawer.

Part B — Defects identified, not yet fixed

3 items

Found in the second review pass. Two of these silently destroy user data.

D1 Undo does not restore subtasks or comments OPEN · HIGH

Deleting a task cascade-deletes its subtasks, comments and activity server-side (`tasks.service.ts:171`). The undo introduced in A5 recreates only the parent task, so **its children are permanently lost** even though the UI implies a full restore. **Suggested:** either a restore endpoint that soft-deletes and revives the whole subtree, or narrower toast wording. The current behaviour promises more than it delivers, which is worse than offering no undo at all.

D2 Bulk delete has no undo and is not atomic OPEN · HIGH

The column header menu offers “Delete all (N)”, which loops over tasks firing a separate DELETE per card. Nothing confirms, nothing undoes, and a failure halfway leaves the column partially deleted with no indication of which succeeded. **Suggested:** a bulk endpoint deleting in one operation, with the same undo affordance as single deletes.

D3 No optimistic-concurrency control OPEN · MEDIUM

Two people editing the same task both PATCH it and the last write silently wins — the first change disappears with no warning. `updatedAt` is already returned but never checked on write. **Suggested:** accept a version field and return 409 Conflict on mismatch. Worth doing only if multi-user editing is a real scenario for this product.

Part C — UX gaps identified

5 items

Rough edges that do not lose data but noticeably affect the experience.

U1 Errors offer no way to recover OPEN · MEDIUM

A failed fetch renders its message as static text with no retry, so the only recovery is a full page reload. **Suggested:** a “Try again” button calling the existing `reload()` — the hook already exposes it, making this a small change with real payoff on flaky connections.

U2 Empty states are a bare sentence OPEN · MEDIUM

An empty board shows only “No tasks yet.” — no illustration, no explanation, no call to action, on a new user’s very first screen. **Suggested:** pair it with a primary “Add your first task” button, and offer “Clear filters” on the filtered variant.

U3 View mode and filters do not survive a reload OPEN · MEDIUM

Board/List choice, active filters and column visibility are all component state, so a refresh resets everything. **Suggested:** persist view and field visibility to `localStorage`, and reflect search and filters in the URL query string — which also makes a filtered view shareable.

U4 **No custom 404 or error boundary** OPEN · MEDIUM

A non-existent task shows Next.js's default not-found page, carrying none of the app's design; an uncaught render error shows a blank page in production. **Suggested:** `app/not-found.tsx` and `app/error.tsx` using the app shell, the latter with a reset button.

U5 **Task titles have no length feedback** OPEN · LOW

The API rejects titles over 200 characters, but no input enforces or signals the limit, so a long title is accepted by the field and rejected on save. **Suggested:** `maxLength` plus a counter as the limit approaches.

Part D — Polish and hardening

4 items

Lower-urgency work, plus what should be done before the deployed link circulates.

P1 **Optimistic UI** DEFERRED

Every change round-trips before the UI updates — imperceptible locally, sluggish on a slow connection. **Not implemented deliberately:** optimistic updates only help with proper rollback, and a half-built layer that silently diverges from the database is worse than the current behaviour, which is correct if not fastest.

P2 **Board columns do not scroll independently** OPEN · LOW

Each column grows with its content, so one long column stretches the board and pushes the others' "Add Task" far down the page. **Suggested:** a `max-height` with per-column overflow.

H1 **No rate limiting on guest login** OPEN · HIGH

`POST /api/auth/guest` writes a user document per call and is unauthenticated, making it trivially abusable on a public URL. **This is already measurable:** the development database holds **80 users, 70 of them guests (88%)** — accumulated from testing alone. A free M0 cluster would fill quickly under real traffic. **Suggested:** `@nestjs/throttler` on the auth routes plus a scheduled job pruning old guest accounts; neither exists today.

H2 Missing security headers; pagination unused

[OPEN](#) • [MEDIUM](#)

No helmet — a one-line addition supplying standard security headers. **Pagination exists but is unused** — the API supports `skip/take` capped at 100, but the UI never paginates, so a board past 100 tasks silently truncates with no indication rows are missing.

#	IMPROVEMENT	SEVERITY	STATUS
A1	Comments heading mislabelled	Defect	Implemented
A2	Board drag-and-drop + keyboard equivalent	UX gap	Implemented
A3	All seven filter axes functional	UX gap	Implemented
A4	Duplicate “Members” label	UX gap	Implemented
A5	Undo on delete	UX gap	Implemented
A6	Mutation failures surfaced	UX gap	Implemented
A7	Loading skeletons	Polish	Implemented
A8	Focus rings + drawer focus trap	Polish	Implemented
D1	Undo loses subtasks and comments	Defect	Open — high
D2	Bulk delete: no undo, not atomic	Defect	Open — high
D3	No optimistic-concurrency control	Defect	Open — medium
U1	Errors offer no retry	UX gap	Open — medium
U2	Empty states lack a call to action	UX gap	Open — medium
U3	View and filters lost on reload	UX gap	Open — medium
U4	No custom 404 or error boundary	UX gap	Open — medium
U5	No title length feedback	UX gap	Open — low
P1	Optimistic UI	Polish	Deferred
P2	Board columns do not scroll	Polish	Open — low
H1	No rate limiting on guest login	Hardening	Open — high

H2	No helmet; pagination unused	Hardening	Open — medium
----	------------------------------	-----------	------------------

All implemented work was verified against the running application: the existing interaction suite passes **21/21**, a dedicated feature suite passes **22/22**, and the responsive audit is clean across **ten widths from 320px to 1920px** — with zero console errors throughout.

Priority if this were continuing: D1 and D2 first, since both silently destroy user data, then H1 before the deployed link circulates widely.

14

Known limitations

Design comparison against the Figma source was not performed. The Figma file for this assessment could not be opened with the access available during development. This document therefore describes the implementation as built and verified; it makes no claim of pixel-level equivalence to the Figma design, and no measurement audit was carried out.

Some detail-header controls are presentational. Lock, watcher count and share render per the design but are not wired to behaviour.

Atlas cold start takes roughly 20 seconds. Mongoose logs one retry before connecting. Harmless locally, but a platform with a short startup healthcheck could fail on it.

No unit test suite. `jest` finds no `.spec.ts` files; coverage comes from the end-to-end harnesses. `npm test` in the backend exits non-zero on an empty suite.

15

Running locally

```
# Backend – needs backend/.env with DATABASE_URL and JWT_SECRET
cd backend
npm install
npm run build
npm run start:prod           # http://localhost:4000/api

# Frontend – needs frontend/.env.local with NEXT_PUBLIC_API_URL
cd frontend
npm install
npm run build
npm start                   # http://localhost:3000
```

Both `.env.example` files are committed as templates. To run the interaction suite with both servers up:

```
cd frontend
npm i -D playwright && npx playwright install chromium
APP_URL=http://localhost:3000 node test/ui-check.mjs
```

Pyramid — walkthrough documentation. Screenshots captured from the running application at 1440×900, device scale 2. All verification figures reflect actual command output.